Phuthiphong Wongchantib 660610840

CounterClockwise-Only Steering Robot — Detour Steer Algorithm

Drive on, steer less, come back

Robot. One DC motor: turning it one way *steers* the wheel module, *counter-clockwise only* (one-way clutch); turning it the other way *drives* forward. It can never steer and drive at the same time. The old method steers straight at the goal, so a goal a little to the *right* (φ close to 360°) costs almost a full turn. **Idea.** Two straight legs whose headings differ by more than 180° can reach *any* direction, so the robot may instead drive a straight on, steer only β^* ($180^\circ < \beta^* < \varphi$), and drive b to the goal. The best β^* has a closed form. Headings are counter-clockwise from the $+x$ axis. ω steer speed, v drive speed, t_s settle time, t_b stop time (drive → steer), tol tolerance, r_{goal} arrival radius. **On the robot** (`mor_luam`, ESP32) the module turns counter-clockwise seen from above: the same frame as this page. The firmware helper name cwErrorDeg is historical; its sign is counter-clockwise. Algorithms 1.1–1.5 plan; 1.6–1.7 are how the robot steers and drives each leg.

Algorithm 1.1: Counter-Clockwise Heading Helpers

Procedure : *Bearing*(Δx, Δy) // CounterClockwise heading that points along $(\Delta x, \Delta y)$
  return $\operatorname{atan2}(\Delta y, \Delta x) \bmod 2\pi$;
end

Procedure : *SteeringError*(θ, h) // How far the wheel must turn counter-clockwise: $0 \ldots 2\pi$
  return $(\theta - h) \bmod 2\pi$;
end

Procedure : *InTolerance*(e) // Close enough on either side of the target
  return $e \le tol$ **or** $e \ge 2\pi - tol$; // Just *past* the target reads as $e \approx 2\pi$
end

Algorithm 1.2: Optimal Detour Angle in Closed Form, $O(1)$

Procedure : *OptimalDetourAngle*(φ, d) // φ: counter-clockwise angle to the goal, d: distance
  if $\varphi \le \pi$ **then return None**; // Theorem 1: steering straight is optimal
  $k \leftarrow (d\,\omega / v)\,|\sin\varphi|$; // One number decides everything
  if $k \ge 2$ **then return None**; // Theorem 3: time keeps falling up to $\beta = \varphi$
  $\beta^* \leftarrow 2\pi - \arccos(k - 1)$; // The only root of $dT/d\beta = 0$ in $(\pi, 2\pi)$
  if $\beta^* \ge \varphi$ **then return None**; // The best angle is not a detour
  return β^*;
end

Algorithm 1.3: Plan One Leg: Direct or Detour, $O(1)$

Procedure : *PlanLeg*(φ, d) // Returns (kind, a, β, b, time)
  $T_D \leftarrow \varphi/\omega + t_s + d/v$; // Old method: steer φ, drive d
  $\beta^* \leftarrow$ *OptimalDetourAngle*(φ, d);
  if $\beta^* =$ **None then return** (DIRECT, 0, φ, d, T_D);
  $a \leftarrow d \sin(\beta^* - \varphi) / \sin\beta^*$; // Leg 1: heading unchanged, no steer
  $b \leftarrow d \sin\varphi / \sin\beta^*$; // Leg 2: heading β^*, ends at the goal
  $T_R \leftarrow a/v + t_b + \beta^*/\omega + t_s + b/v$; // One extra stop between the legs
  if $T_R < T_D$ **then return** (DETOUR, a, β^*, b, T_R); // Use the detour only if it is really faster
  return (DIRECT, 0, φ, d, T_D);
end

Algorithm 1.4: Closed-Loop Detour Steer to One Goal (plan again after every move)

```
Procedure : DetourSteer(p, h, g)                          // p: position, h: wheel heading, g: goal
    while ‖g − p‖ > r_goal do
        d ← ‖g − p‖;    e ← SteeringError(Bearing(g − p), h);
        if InTolerance(e) then                            // Already aimed: drive to the goal
            DriveReaim(d);                                // Algorithm 1.7
        else
            plan ← PlanLeg(e, d);
            if plan.kind = DETOUR then
                DriveReaim(plan.a);                       // Leg 1: straight on, no steer
                Stop();                                   // t_b: switch the motor from drive to steer
            else
                SteerOneWay(Bearing(g − p));              // Algorithm 1.6: stops a little short on purpose
            end
        end
        p ← MeasurePosition();    h ← MeasureHeading();   // AS5600 wheel angle + IMU body heading; T4
        // Short by ε: e = ε, a small extra steer. Past by ε: e = 2π − ε, PlanLeg detours
    end
    return (p, h);
end
```

Algorithm 1.5: End-to-End Route Following with Detour Steer

```
Procedure : FollowRouteDetour(W)                          // Master procedure: visit the goals W in order
    p ← (0, 0);    h ← 0;                                 // Start pose
    foreach g ∈ W do
        (p, h) ← DetourSteer(p, h, g);
    end
    Display total time;        // Goal 0.30 m at 354°: a=0.31 m, β*=249°, b=0.034 m, 18.83 s (old method: 20.16 s)
end
```

Algorithm 1.6: Steer One Way: Stop a Little Short on Purpose (robot, every 10 ms)

```
Procedure : SteerOneWay(θ)                                // θ: the heading to reach
    while true do                                         // Every control tick, 10 ms
        h ← MeasureHeading();                             // Wheel (AS5600) + body (IMU): the body twists as it steers
        e ← SteeringError(θ, h);    r ← steering speed over 50 ms;
        if e < π and e ≤ e_land + r c then break;         // Cut early: it coasts on to ≈ e_land short
        if InTolerance(e) and (e > π or r < r_min) then break;  // Past, or standing: a push from rest jumps past
        steering motor on;
    end
    steering motor off; wait until the wheel stops and the body turns < 5°/s (at most 0.4 s);
    c ← (1 − λ) c + λ (angle coasted / r_cut);            // Learn the coast time c
end
```

Algorithm 1.7: Drive with Early Re-aim (one motor: it cannot steer while it drives)

```
1  Procedure : DriveReaim(ℓ)                          // ℓ: leg length on the current heading h
2      q ← p + ℓ (cos h, sin h);                      // The end of this leg
3      while true do                                  // Every control tick, 10 ms
4          (f, s) ← q − p seen from the wheel;        // f ahead, s beside (+: steering side)
5          if ‖q − p‖ ≤ r_goal or f ≤ 0 then break;   // Arrived, or level with it
6          if s > r_goal and f ≤ max(m, R s) then break;   // Re-aim is then only arctan(1/R)
7          drive motor on;
8      end
9      drive motor off;    p ← MeasurePosition();     // Algorithm 1.4 plans again from here
10 end
```

Correctness, complexity, results and closest prior work

For a goal at counter-clockwise angle $\varphi > \pi$ and distance d, a detour with final heading β takes

$$T(\beta) = \frac{\beta}{\omega} + t_s + t_b + \frac{d}{v}\, g(\beta), \qquad g(\beta) = \frac{\sin(\beta - \varphi) + \sin\varphi}{\sin\beta}, \qquad g'(\beta) = \frac{\sin\varphi}{1 + \cos\beta}.$$

T1. If $\varphi \le 180°$, steering straight at the goal is optimal. Two headings less than 180° apart only reach directions between them, so every path steers at least φ; every path also drives at least d. The direct path does both.

T2. If $\varphi > 180°$, a best path has at most 2 legs, with headings 0 and β. The shortest way to write the goal vector with headings in $[0, \beta]$ uses the two headings on either side of φ (the optimum of a 2-row linear program has 2 non-zero variables).

T3. On $(\pi, 2\pi)$, $T'(\beta) = 1/\omega - (d/v)|\sin\varphi|/(1 + \cos\beta)$ is strictly increasing, so T has exactly one minimum: $\cos\beta^* = k - 1$ with $k = (d\omega/v)|\sin\varphi|$. If $k \ge 2$, T keeps falling up to $\beta = \varphi$, which is the direct path.

T4. After leg 1, *PlanLeg(β^*, b)* returns "steer β^*, drive b". Any other detour from there is a longer leg 1 plus one more stop t_b, which T3 already beat. So the planner can run again after every move (closed loop) without changing its mind.

O. *PlanLeg()* is $O(1)$ in time and memory: 0.52 µs per call in Python, against 505 µs for a 3600-point search over β (973×). T1–T4 are also checked by brute force in `detour_steer.py`: closed form vs a 20 000-point grid, vs every 2-leg path on a 2° grid, and 2000 replans. All PASS.

Unknown stop error. Nobody knows in advance whether the wheel stops a little past or a little short. The robot reads the real wheel angle (AS5600) and body heading (IMU) after it stops, and *DetourSteer()* plans again from there; *PlanLeg()* never uses the error. Inside $\pm tol$ the robot drives on its real heading, and a miss of more than r_{goal} is planned again.

Why stop short on purpose (Algorithm 1.6). Past its angle by more than *tol*, a one-way wheel needs almost a full turn; short of it, only a little more. So the robot cuts the motor early and lets the wheel coast to $e_{land} = 2°$ short of the angle (c, the coast time, is learned on every steer). A wheel standing inside the band is never pushed again: from rest the motor must build up power and the wheel jumps past (measured: 9°). Stopping short also means a miss always leaves the goal on the *steering side* of the line the robot drives on, so *DriveReaim()* fixes it with a small turn ($\arctan(1/R) = 18°$ for $R = 3$, $m = 2$ cm) instead of a 90° turn after passing the goal.

Results in simulation, at the robot's own speeds ($v = 0.03$ m/s, $\omega = 35°/s$, $t_b = 0.2$ s, $r_{goal} = 2.5$ mm, the values measured on `mor_luam`; 5000 random goals, $d \in [0.3, 2]$ m; the same goals and stop errors for both; mean time per goal):

wheel stops	bias	sd	old method (s)	detour (s)	saved
exactly	0°	0°	43.40	43.40	0.0 %
either way, random	0°	1°	48.33	48.10	0.5 %
either way, random	0°	2°	50.10	49.60	1.0 %
either way, random	0°	3°	53.00	51.75	2.4 %
mostly short	−2°	2°	48.21	48.08	0.3 %
mostly past	+2°	2°	57.55	55.09	4.3 %
mostly past	+2°	3°	60.44	56.88	5.9 %

At the robot's speeds Detour Steer was slower on 0 % of the goals in every case, but the gain on random goals is small (0–5.9 %). The robot drives slowly, so driving takes most of the time, and a detour is chosen only for goals just to the right: from $\varphi \ge 346°$ at $d = 0.30$ m and $\varphi \ge 356°$ at $d = 1$ m. For the goal of the robot test (0.30 m at 354°) the model gives 18.83 s against 20.16 s (6.6 %, Figure 2a).

Other speeds (the same 5000 goals and stop errors, sd 2°; time saved by Detour Steer, Figure 3):

drive \ steer	20°/s	35°/s	60°/s	120°/s
0.03 m/s (the robot)	2.3 %	1.0 %	0.4 %	0.1 %
0.10 m/s	5.4 %	3.8 %	2.2 %	0.6 %
0.25 m/s	6.7 %	5.7 %	4.2 %	2.2 %

A faster drive or a slower steer makes the detour pay more. At the robot's drive speed with steering at 20°/s, 0.2 % of the goals were a little slower with Detour Steer (a stop error after the extra stop); at 35°/s and faster, none were.

Results on the robot (`mor_luam`: $v = 0.03\,\mathrm{m/s}$, $\omega = 35^\circ/\mathrm{s}$ its measured steering speed, $t_b = 0.2\,\mathrm{s}$, $r_{goal} = 2.5\,\mathrm{mm}$). The same task for both: before the clock starts the wheel is turned to the start heading and is still; the goal is then placed $d = 0.30\,\mathrm{m}$ away at $\varphi = 354^\circ$ from where the wheel really points. The robot times each run itself and drives back to the start (not timed). 3 rounds, order old–detour, detour–old, old–detour:

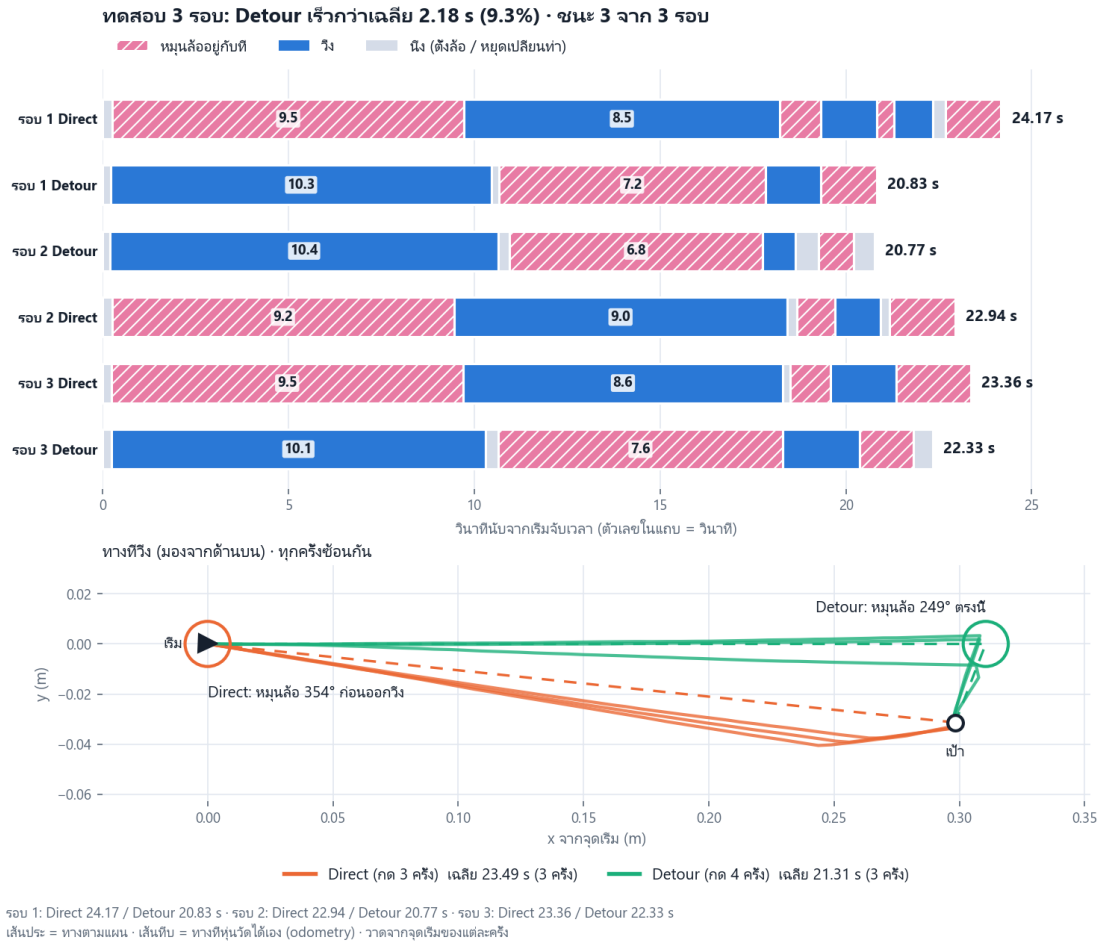
round	old method (s)	Detour Steer (s)	saved (s)
1	24.17	20.83	3.34
2	22.94	20.77	2.17
3	23.36	22.33	1.03
mean	23.49	21.31	2.18 (9.3 %)

Detour Steer was faster in 3 of 3 rounds. It steered 3.8 s less (8.5 vs 12.4 s) and drove 1.2 s more (11.8 vs 10.6 s). The plan predicted 18.8 vs 20.2 s; the floor gave a bigger gap, because the old method's near-full turn also needs more correction at the end. All 6 runs ended within 2.5 mm of the goal.

Why the robot drifts right (both methods). (1) One motor steers and drives, so the robot cannot correct its course while it drives: any drift adds up until it stops and re-aims (Algorithm 1.7). (2) Steering and driving act on each other through the same gear train: the drive torque turns the wheel module and its gears a little (while it drives, the AS5600 wheel angle keeps changing and the front of the robot turns), and a near-full steer twists the body about 4°. (3) The wheel slips, the floor is not flat, and the robot's weight is not balanced. (4) The robot stops steering about 2° short on purpose (past would cost almost a full turn); it steers counter-clockwise, so short means the wheel points a little to the right of the goal. Together the body drifts about 3° while driving (IMU), and both methods end 1–3 cm right of their line; the early re-aim (about 18°) brings them back.

Closest prior work, and the difference. One-motor steer-and-drive mechanisms (US 8,424,627 B2, 2013; Sekine & Mizuuchi, ICINCO 2022) give no planner. Dubins cars that can turn one way only (Bakolas & Tsiotras, JOTA 2011; Patsko & Fedotov, 2018–2019) turn *while moving*. Turn-cost routing (angular-metric and quadratic TSP) orders waypoints, and predictive coast stopping (US 8,571,394 B2) stops early. None of the work found gives this closed-form detour for a robot that must *stop to steer, counter-clockwise only*. "Not found" is not proof that nobody did it.

Figure 1. The 3 rounds on the robot, from its own record



Top: what each run did over time (hatched pink = steering in place, blue = driving, grey = still). The old method first steers $\approx$ 9.4 s in place, then drives; Detour Steer drives at once and steers less later. **Bottom:** the paths the robot measured (odometry), all six from one start, seen from above; dashed = the plan. (The robot steers counter-clockwise, so here the goal is to the right.)

Figure 2. Simulation at the robot's speeds: two goals 0.30 m away

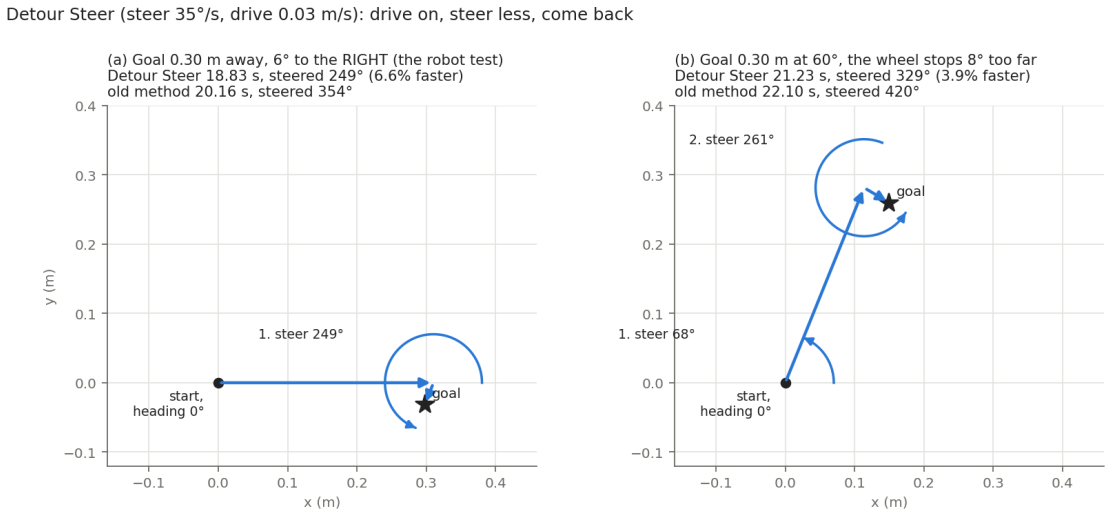


Figure 3. Other speeds: time saved on the same 5000 goals



Simulation against the robot, and the 2 m route

How far the simulation is from the robot. The only route run on both is the robot test (0.30 m at 354°, 3 rounds each). The simulation uses the robot's measured speeds ($v = 0.03$ m/s, $\omega = 35°/s$, $t_b = 0.2$ s, $r_{goal} = 2.5$ mm):

	simulation (s)	robot, mean of 3 (s)	error
old method	20.16	23.49	−3.33 s (−14.2 %)
Detour Steer	18.83	21.31	−2.48 s (−11.6 %)
time saved	1.33 (6.6 %)	2.18 (9.3 %)	

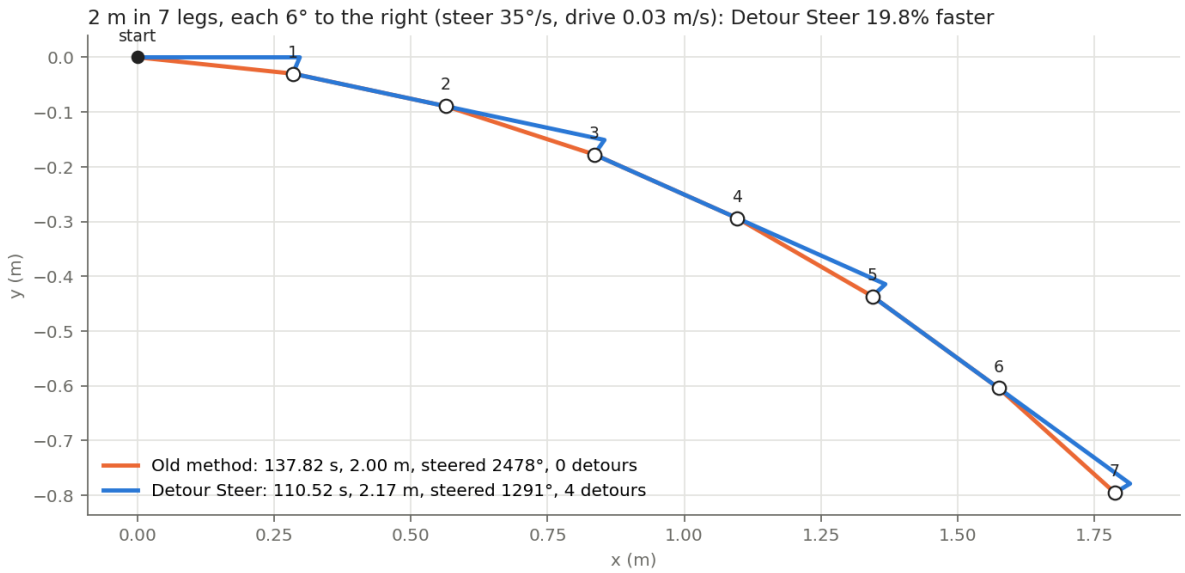
The simulation is 11.6–14.2 % faster than the robot, because it has no drift, no slip and no re-aim stops (see "Why the robot drifts right"). It also under-states the gain: the robot saved 9.3 %, the simulation 6.6 %, because the old method's near-full turn needs more correction on the floor.

The 2 m route (Figure 4). A total of 2 m split into 7 equal legs (2/7 m each), each leg 6° to the right of the one before; both methods visit the same points. At the robot's speeds the simulation gives:

	time (s)	path (m)	steered	detours
old method	137.82	2.00	2478°	0
Detour Steer	110.52	2.17	1291°	4
saved	27.30 (19.8 %)			

Every leg is 6° to the right, so the old method steers about 354° at each point; Detour Steer drives on and steers less at 4 of the 7 points. This route was run in simulation only. If the robot has the same error as in the 0.30 m test (old method −14.2 %, Detour Steer −11.6 %), the robot would take about 160.6 s and 125.1 s (22.1 % faster); this is an estimate from the simulation, not a measurement. The return to the start (about 1.96 m) is not timed.

Figure 4. Simulation of the 2 m / 7-leg route at the robot's speeds



Code and Git repository

The code of this algorithm (Python planner, simulators, and the robot firmware) is on Git:

`https://github.com/Mannaja78520/mor_luam`

Video of the robot working:

`https://drive.google.com/drive/folders/11qOM6UoeH2c4nEuM0a0JO6eXTnr0eDud`

The clip shows the robot working in synchronize mode. There is no clip of the Detour Steer vs old method (Direct) comparison; that comparison comes from the robot's own record (Figure 1 and the table of results on the robot).parmedskip
Branch `main`. Where things are:

- Planner and simulators: `algorithm_sim/`
- Robot version: `firmware/src/algorithm/DetourSteer.cpp`
- How to install and run: `algorithm_sim/README.md` (Docker is the recommended install)